

Unit Testing with JUnit 5 + Mockito

Interview Perspective — Service-Layer Testing, Mocking, and Strategy
September 2026

Unit Testing with JUnit 5 + Mockito — Interview Perspective

1. What Is Unit Testing?
2. Why Unit Testing?
3. JUnit 5 vs Mockito
 - JUnit 5
 - Mockito
4. Basic JUnit 5 Test
5. AAA Pattern
6. Important JUnit 5 Assertions
7. `assertEquals()` vs `assertSame()`
8. Testing Exceptions
9. Mockito — Why Do We Need It?
10. Creating a Mock
11. `@Mock`
12. `@InjectMocks`
13. Don't Confuse `@InjectMocks` With Spring DI
14. `when().thenReturn()`
15. `thenThrow()`
16. `thenAnswer()`
17. `verify()`
18. `verify()` With Call Count
19. `verifyNoInteractions()`
20. Argument Matchers
21. Important Matcher Rule
22. `ArgumentCaptor`
23. Example: Real Service Unit Test
24. What Should You Test?
25. Unit Test vs Integration Test
 - Unit test
 - Integration test
26. `@SpringBootTest` vs Mockito Unit Test
27. Testing Controller Layer
28. Testing Repository Layer
29. Testing External APIs
30. Mock vs Stub vs Spy
 - Mock
 - Stub
 - Spy
31. Mock vs Spy
32. Why Mocking Everything Is Bad
33. Example of a Bad Test
34. Testing Private Methods
35. Test Naming
36. Parameterized Tests
37. `@CsvSource`
38. `@BeforeEach` vs `@BeforeAll`
39. Test Independence
40. What Does Good Unit Test Coverage Mean?

41. Testing Exception + Interaction
42. Mockito Strictness
43. Mocking Static Methods / Constructors
44. Unit Testing a Strategy Pattern
45. A Realistic BPS Testing Strategy
46. What Should You Mock in BPS?
47. Example Interview Question
48. Another Important Interview Question
49. Mockito Verification vs Assertion
50. Testing Async Code
51. Test Doubles — Know the Terminology
52. Common Mistakes
53. The Testing Pyramid
54. Interview-Ready Answer: “How Do You Unit Test a Spring Service?”
55. Interview Questions You Should Be Able to Answer
 - JUnit 5
 - Mockito
 - Spring Boot
56. The One Architecture You Should Remember

Unit Testing with JUnit 5 + Mockito — Interview Perspective

For a **3+ year Java/Spring Boot backend role**, you should be able to do more than write `@Test`.

An interviewer will typically test whether you understand:

```
Unit testing
↓
What should be tested?
↓
What should be mocked?
↓
JUnit 5 lifecycle/assertions
↓
Mockito behavior/verification
↓
Service-layer testing
↓
Exception & edge-case testing
↓
Spring Boot testing strategy
```

The most important distinction:

JUnit 5 provides the testing framework. Mockito provides mock objects and interaction testing.

1. What Is Unit Testing?

A unit test tests a **small, isolated unit of behavior**, usually a method/class, without depending on external systems.

For example:

```
@Service
public class PaymentService {

    private final PaymentRepository repository;

    public PaymentService(PaymentRepository repository) {
        this.repository = repository;
    }

    public Payment getPayment(Long id) {
        return repository.findById(id)
            .orElseThrow(() ->
                new PaymentNotFoundException(id));
    }
}
```

You want to test:

```
PaymentService
↓
business logic
```

without actually connecting to:

```
Database
External API
```

```
Kafka
Redis
File system
```

So:

```
PaymentService
└─ PaymentRepository → MOCK
```

2. Why Unit Testing?

A good interview answer:

“

Unit testing helps verify individual units of business logic in isolation. It provides fast feedback, catches regressions, documents expected behavior, and allows developers to safely refactor code. External dependencies such as databases or external APIs are usually mocked so that the test focuses on the unit's own behavior.

3. JUnit 5 vs Mockito

This distinction is fundamental.

JUnit 5

Provides:

```
@Test
@BeforeEach
@AfterEach
@BeforeAll
@AfterAll

Assertions
@TestMethodOrder
@ParameterizedTest
@Nested
@Tag
```

Example:

```
@Test
void shouldCalculateTotal() {

    int result = calculator.add(2, 3);

    assertEquals(5, result);
}
```

Mockito

Provides:

```
mock
when
thenReturn
verify
```

```
ArgumentCaptor  
spy  
@InjectMocks  
@Mock
```

Example:

```
PaymentRepository repository =  
    mock(PaymentRepository.class);  
  
when(repository.findById(1L))  
    .thenReturn(Optional.of(payment));
```

So:

```
JUnit  
→ runs/tests/asserts  
  
Mockito  
→ creates test doubles and controls/verifies dependencies
```

4. Basic JUnit 5 Test

Production code:

```
public class Calculator {  
  
    public int add(int a, int b) {  
        return a + b;  
    }  
}
```

Test:

```
class CalculatorTest {  
  
    private Calculator calculator;  
  
    @BeforeEach  
    void setUp() {  
        calculator = new Calculator();  
    }  
  
    @Test  
    void shouldAddTwoNumbers() {  
  
        int result = calculator.add(10, 20);  
  
        assertEquals(30, result);  
    }  
}
```

5. AAA Pattern

A very common testing convention:

```
Arrange  
Act  
Assert
```

Example:

```
@Test
void shouldCalculatePaymentAmount() {

    // Arrange
    Payment payment = new Payment();
    payment.setAmount(new BigDecimal("500"));

    // Act
    BigDecimal result =
        service.calculate(payment);

    // Assert
    assertEquals(
        new BigDecimal("500"),
        result
    );
}
```

Think:

```
Arrange → setup
Act      → execute
Assert   → verify
```

This is a good default structure for your tests.

6. Important JUnit 5 Assertions

Know these well:

```
assertEquals(expected, actual);
assertNotEquals(expected, actual);

assertTrue(condition);
assertFalse(condition);

assertNull(value);
assertNotNull(value);

assertSame(expectedObject, actualObject);
assertNotSame(expectedObject, actualObject);

assertThrows(Exception.class, () -> ...);

assertDoesNotThrow(() -> ...);
```

7. assertEquals() VS assertEquals()

Common interview question.

assertEquals

Checks logical equality:

```
assertEquals(expected, actual);
```

uses equals() semantics.

assertSame

Checks whether they are the **same object reference**:

```
assertSame(expected, actual);
```

Conceptually:

```
equals  
→ same value/equality  
  
same  
→ same object in memory
```

8. Testing Exceptions

Suppose:

```
public Payment getPayment(Long id) {  
  
    return repository.findById(id)  
        .orElseThrow(() ->  
            new PaymentNotFoundException(id));  
}
```

Test:

```
@Test  
void shouldThrowWhenPaymentDoesNotExist() {  
  
    when(repository.findById(100L))  
        .thenReturn(Optional.empty());  
  
    PaymentNotFoundException exception =  
        assertThrows(  
            PaymentNotFoundException.class,  
            () -> service.getPayment(100L)  
        );  
  
    assertEquals(  
        "Payment not found: 100",  
        exception.getMessage()  
    );  
}
```

This is much better than:

```
try {  
    service.getPayment(100L);  
    fail();  
} catch (...) {  
}
```

9. Mockito — Why Do We Need It?

Suppose your service does:

```
PaymentService  
↓  
PaymentRepository  
↓
```

Database

A unit test should not need a real database.

So Mockito gives us:

```
PaymentRepository
  ↓
  MOCK
```

You control what it returns.

10. Creating a Mock

Without annotations:

```
PaymentRepository repository =
    Mockito.mock(PaymentRepository.class);
```

Then:

```
when(repository.findById(1L))
    .thenReturn(Optional.of(payment));
```

Now whenever the test calls:

```
repository.findById(1L)
```

Mockito returns your predefined result.

11. @Mock

Usually cleaner:

```
@Mock
private PaymentRepository repository;
```

But the Mockito annotations need initialization.

With JUnit 5:

```
@ExtendWith(MockitoExtension.class)
class PaymentServiceTest {

    @Mock
    PaymentRepository repository;

}
```

This is the preferred pure-unit-test setup.

12. @InjectMocks

Suppose:

```

@Service
public class PaymentService {

    private final PaymentRepository repository;

    public PaymentService(
        PaymentRepository repository) {
        this.repository = repository;
    }
}

```

Test:

```

@ExtendWith(MockitoExtension.class)
class PaymentServiceTest {

    @Mock
    PaymentRepository repository;

    @InjectMocks
    PaymentService service;
}

```

Mockito creates the `PaymentService` and injects the mock repository.

Conceptually:

```

repository mock
  ↓
PaymentService
  ↓
@InjectMocks

```

13. Don't Confuse `@InjectMocks` With Spring DI

This is important.

```
@InjectMocks
```

is a **Mockito annotation**.

It does not start Spring.

It doesn't mean:

```
Spring ApplicationContext
```

is running.

It's simply Mockito constructing the class under test and injecting available mocks/spies.

Therefore:

```
@ExtendWith(MockitoExtension.class)
```

is appropriate for a pure unit test.

14. `when().thenReturn()`

The most common Mockito syntax:

```
when(repository.findById(1L))
    .thenReturn(Optional.of(payment));
```

Meaning:

“
When this method is called with this argument, return this value.

Another example:

```
when(repository.save(any(Payment.class)))
    .thenReturn(payment);
```

15. `thenThrow()`

Test failures/exceptions:

```
when(repository.findById(1L))
    .thenThrow(new RuntimeException("DB failure"));
```

Then:

```
assertThrows(
    RuntimeException.class,
    () -> service.getPayment(1L)
);
```

This lets you test how your service handles dependency failures.

16. `thenAnswer()`

Sometimes you need dynamic behavior.

```
when(repository.save(any(Payment.class)))
    .thenAnswer(invocation ->
        invocation.getArgument(0));
```

Meaning:

“
Return the same Payment object passed to `save()` .

Useful when the mock behavior depends on the input.

But don't use `thenAnswer()` when simple `thenReturn()` is sufficient.

17. `verify()`

This is extremely important.

Suppose:

```
service.processPayment(payment);
```

You want to verify that the repository was called:

```
verify(repository)
    .save(payment);
```

This tests an **interaction** with the dependency.

18. `verify()` With Call Count

```
verify(repository, times(1))
    .save(payment);
```

Other options:

```
verify(repository, never())
    .deleteById(anyLong());

verify(repository, times(2))
    .save(any());

verify(repository, atLeastOnce())
    .save(any());
```

19. `verifyNoInteractions()`

Suppose invalid input should prevent any database call:

```
@Test
void shouldNotCallRepositoryForInvalidPayment() {

    assertThrows(
        IllegalArgumentException.class,
        () -> service.process(null)
    );

    verifyNoInteractions(repository);
}
```

This is a useful test of a business rule.

20. Argument Matchers

Mockito provides matchers:

```
any()
anyString()
anyLong()
anyInt()
eq(..)
isNull()
notNull()
```

Example:

```
when(repository.findById(anyLong()))
```

```
.thenReturn(Optional.of(payment));
```

Or:

```
verify(repository)
    .findById(eq(100L));
```

21. Important Matcher Rule

Don't casually mix raw values and matchers.

Bad:

```
when(service.process(
    "PAYMENT",
    anyLong()
));
```

You are mixing:

```
"PAYMENT" → raw value
anyLong() → matcher
```

Use:

```
when(service.process(
    eq("PAYMENT"),
    anyLong()
));
```

General rule:

“ When using argument matchers in a method call, use matchers for the arguments consistently.

22. ArgumentCaptor

Very useful interview topic.

Suppose:

```
service.processPayment(request);
```

and internally:

```
repository.save(payment);
```

You want to inspect exactly what was passed to `save()`.

Use:

```
ArgumentCaptor<Payment> captor =
    ArgumentCaptor.forClass(Payment.class);

verify(repository).save(captor.capture());

Payment savedPayment =
    captor.getValue();
```

```
assertEquals(  
    PaymentStatus.PAID,  
    savedPayment.getStatus()  
);
```

This is especially useful when the service **creates/transforms an object internally**.

23. Example: Real Service Unit Test

Production code:

```
@Service  
public class PaymentService {  
  
    private final PaymentRepository repository;  
  
    public PaymentService(  
        PaymentRepository repository) {  
        this.repository = repository;  
    }  
  
    public Payment process(Long id) {  
  
        Payment payment =  
            repository.findById(id)  
                .orElseThrow(() ->  
                    new PaymentNotFoundException(id));  
  
        if (payment.getAmount()  
            .compareTo(BigDecimal.ZERO) <= 0) {  
            throw new IllegalArgumentException(  
                "Invalid amount");  
        }  
  
        payment.setStatus(PaymentStatus.PAID);  
  
        return repository.save(payment);  
    }  
}
```

Test:

```
@ExtendWith(MockitoExtension.class)  
class PaymentServiceTest {  
  
    @Mock  
    PaymentRepository repository;  
  
    @InjectMocks  
    PaymentService service;  
  
    @Test  
    void shouldProcessPayment() {  
  
        Payment payment = new Payment();  
        payment.setId(1L);  
        payment.setAmount(  
            new BigDecimal("500"));  
        payment.setStatus(  
            PaymentStatus.PENDING);  
  
        when(repository.findById(1L))  
            .thenReturn(Optional.of(payment));  
  
        when(repository.save(payment))
```

```

        .thenReturn(payment);

        Payment result =
            service.process(1L);

        assertEquals(
            PaymentStatus.PAID,
            result.getStatus()
        );

        verify(repository)
            .findById(1L);

        verify(repository)
            .save(payment);
    }
}

```

This is a classic service-layer unit test.

24. What Should You Test?

Don't test every line mechanically.

Test **behavior**.

For the above service:

Happy path

```

Payment exists
+
valid amount
→ status becomes PAID
→ save called

```

Payment doesn't exist

```

repository returns empty
→ PaymentNotFoundException
→ save never called

```

Invalid amount

```

amount <= 0
→ IllegalArgumentException
→ save never called

```

Repository failure

```

repository throws exception
→ verify expected handling/propagation

```

This is much more valuable than chasing 100% line coverage.

25. Unit Test vs Integration Test

This distinction is **very important**.

Unit test

```
PaymentService
  ↓
Mockito mock
  ↓
PaymentRepository
```

No real DB.

Usually:

```
@ExtendWith(MockitoExtension.class)
```

Integration test

```
Spring Boot
  ↓
Service
  ↓
Repository
  ↓
Database
```

Potentially:

```
@SpringBootTest
```

or a more focused Spring test.

Integration testing checks that components work together.

26. @SpringBootTest vs Mockito Unit Test

Unit test

```
@ExtendWith(MockitoExtension.class)
```

Advantages:

```
Fast
Isolated
No Spring startup
No database
Easy failure diagnosis
```

@SpringBootTest

```
@SpringBootTest
```

Starts the Spring application context.

Advantages:

```
Tests actual Spring configuration
Dependency wiring
Beans
Configuration
Integration behavior
```

But:

Slower
More dependencies
More setup

Don't use `@SpringBootTest` for every test.

27. Testing Controller Layer

Spring provides:

```
@WebMvcTest(PaymentController.class)
```

This is a **slice test**.

Conceptually:

```
Controller
  ↓
MockMvc
  ↓
Mock Service
```

Example:

```
@WebMvcTest(PaymentController.class)
class PaymentControllerTest {

    @Autowired
    MockMvc mockMvc;

    @MockBean
    PaymentService service;

}
```

Depending on your Spring Boot/Spring version, note that `@MockBean` has been deprecated in newer Spring Boot lines in favor of Spring Framework's `@MockitoBean`. For a Java 17-era codebase, check the actual Spring Boot version before choosing the annotation.

28. Testing Repository Layer

Don't use Mockito to "test" Spring Data itself.

For repository integration tests, commonly use:

```
@DataJpaTest
```

Conceptually:

```
Repository
  ↓
Hibernate
  ↓
Test database
```

This is useful for testing:

- custom JPQL
- native queries

- mappings
- relationships
- persistence behavior

29. Testing External APIs

Suppose your service calls Hydra/Chase or another external API.

Don't call the real external system in a unit test.

Instead:

```
PaymentService
  ↓
PaymentClient
  ↓
MOCK
```

Example:

```
@Mock
PaymentClient paymentClient;
```

Then:

```
when(paymentClient.process(any()))
    .thenReturn(successResponse);
```

Now you can test your business logic deterministically.

For integration/contract testing, a different strategy can be used.

30. Mock vs Stub vs Spy

Interviewers love this.

Mock

Primarily used to:

- control dependency behavior
- verify interactions

```
@Mock
PaymentRepository repository;
```

Stub

A test double configured to return predetermined values.

```
when(repository.findById(1L))
    .thenReturn(Optional.of(payment));
```

You are essentially stubbing the repository.

Spy

Wraps a real object and allows selective stubbing/verification.

```
@Spy
PaymentCalculator calculator;
```

Unlike a pure mock, real methods execute by default.

31. Mock vs Spy

Mock
→ fake object
→ methods don't execute real implementation by default

Spy
→ real object wrapped by Mockito
→ real methods execute by default
→ selected methods can be stubbed

Example:

```
@Spy
PaymentCalculator calculator;
```

Then:

```
doReturn(100)
    .when(calculator)
    .calculate(...);
```

Use spies carefully. Excessive spying often indicates that the class is difficult to test or has too many responsibilities.

32. Why Mocking Everything Is Bad

Suppose:

```
@Mock
Validator validator;

@Mock
Calculator calculator;

@Mock
Mapper mapper;

@Mock
Repository repository;

@Mock
Logger logger;

@Mock
Utility utility;
```

and the test verifies:

```
method A called
```

```
method B called
method C called
method D called
...
```

You may end up testing the **implementation structure**, not the business behavior.

Then a harmless refactoring breaks dozens of tests.

Better:

“
Mock external dependencies that the unit genuinely collaborates with, and assert the observable behavior of the unit.

33. Example of a Bad Test

Production:

```
public Payment process(Payment p) {
    p.setStatus(PAID);
    return repository.save(p);
}
```

Bad test:

```
verify(repository, times(1))
    .save(p);

verify(repository, times(1))
    .save(any());

verify(repository, times(1))
    .findById(...);
```

If the implementation changes but behavior remains correct, the test may break unnecessarily.

Focus on:

```
assertEquals(PAID, result.getStatus());
```

and only verify interactions that are actually important to the behavior.

34. Testing Private Methods

Don't normally test private methods directly.

Bad:

```
testPrivateValidateMethod()
```

Instead test through the public behavior:

```
service.process(payment);
```

If private logic is sufficiently complex that you desperately need to test it independently, that may indicate the logic belongs in a separate class.

For example:

```
PaymentService
  ↓
PaymentValidator
```

Now:

```
PaymentValidatorTest
```

can directly test validation behavior.

35. Test Naming

Prefer descriptive names.

Bad:

```
@Test
void test1() {}
```

Better:

```
@Test
void shouldThrowExceptionWhenPaymentDoesNotExist() {}
```

Another common style:

```
shouldX_whenY
```

Example:

```
void shouldRejectPayment_whenAmountIsNegative()
```

This makes test failures self-explanatory.

36. Parameterized Tests

JUnit 5 supports parameterized testing.

Instead of:

```
test zero
test negative
test 100
test 500
```

you can do:

```
@ParameterizedTest
@ValueSource(ints = {0, -1, -100})
void shouldRejectInvalidAmount(int amount) {

    assertThrows(
        IllegalArgumentException.class,
        () -> validator.validate(amount)
    );
}
```

Very useful for validation/business rules.

37. @CsvSource

For more complex inputs:

```
@ParameterizedTest
@CsvSource({
    "100, 10, 110",
    "200, 20, 220",
    "500, 50, 550"
})
void shouldCalculateTotal(
    int amount,
    int fee,
    int expected) {

    assertEquals(
        expected,
        calculator.total(amount, fee)
    );
}
```

This is a good way to reduce repetitive tests.

38. @BeforeEach VS @BeforeAll

@BeforeEach

Runs before every test:

```
@BeforeEach
void setup() {}
```

@BeforeAll

Runs once before all tests in the class:

```
@BeforeAll
static void setupAll() {}
```

Typically:

```
@BeforeAll
↓
Test 1 setup → Test 1
Test 2 setup → Test 2
Test 3 setup → Test 3
↓
@AfterAll
```

Use shared setup carefully; excessive shared mutable state can make tests order-dependent.

39. Test Independence

Tests should ideally be:

- Independent
- Repeatable
- Deterministic
- Fast

Isolated

A test should not depend on:

another test having run first

For example:

```
@Test
void testA() {
    sharedList.add("A");
}

@Test
void testB() {
    assertEquals(1, sharedList.size());
}
```

Bad design.

Each test should establish its own state.

40. What Does Good Unit Test Coverage Mean?

Don't say:

“
“We need 100% coverage.”

A better answer:

“
Code coverage is a useful signal, but high line coverage does not guarantee good tests. I prioritize important business paths, edge cases, failure scenarios, and branches with meaningful risk.

For a payment service, prioritize:

```
Successful payment
Invalid payment
Payment not found
External service failure
Timeout
Duplicate request
Invalid vendor
Persistence failure
```

rather than blindly maximizing a percentage.

41. Testing Exception + Interaction

Suppose:

```
public Payment process(Long id) {
    Payment p = repository.findById(id)
        .orElseThrow(PaymentNotFoundException::new);
    return repository.save(p);
}
```

Test:

```
@Test
void shouldNotSaveWhenPaymentDoesNotExist() {

    when(repository.findById(1L))
        .thenReturn(Optional.empty());

    assertThrows(
        PaymentNotFoundException.class,
        () -> service.process(1L)
    );

    verify(repository, never())
        .save(any());
}
```

This is a strong test because it verifies both:

Observable result
+
important side effect

42. Mockito Strictness

Modern Mockito/JUnit integrations can help detect unnecessary stubbing.

For example:

```
when(repository.findById(1L))
    .thenReturn(...);
```

but your test never calls:

```
findById(1L)
```

That stubbing may be unnecessary.

Avoid creating huge setup blocks containing mocks/stubs that aren't relevant to the test.

Good test:

Arrange only what this scenario needs.

43. Mocking Static Methods / Constructors

Modern Mockito supports some advanced mocking capabilities, but don't jump to them.

If you find yourself needing:

```
mock static everywhere
mock constructors
mock private methods
```

ask:

“
Is the production code too tightly coupled?”

For example:

```
Payment payment =
    SomeUtility.createPayment();
```

might be difficult to test.

Dependency injection often produces cleaner designs:

```
PaymentFactory factory;

factory.createPayment();
```

Now Mockito can mock the dependency normally.

44. Unit Testing a Strategy Pattern

This is particularly relevant to your payment-domain preparation.

Suppose:

```
interface PaymentProcessor {
    PaymentResult process(PaymentRequest request);
}
```

Test Hydra implementation separately:

```
@ExtendWith(MockitoExtension.class)
class HydraPaymentProcessorTest {

    @Mock
    HydraClient client;

    @InjectMocks
    HydraPaymentProcessor processor;

    @Test
    void shouldProcessSuccessfulPayment() {

        when(client.process(any()))
            .thenReturn(successResponse);

        PaymentResult result =
            processor.process(request);

        assertEquals(
            SUCCESS,
            result.getStatus()
        );

        verify(client).process(any());
    }
}
```

Then test Chase independently.

The Factory gets its own tests.

This gives you:

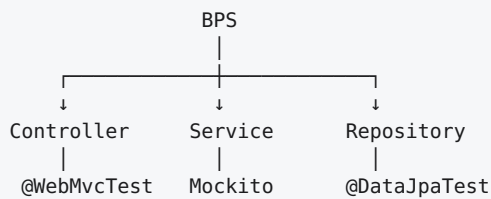
```
PaymentProcessor
├── HydraTest
└── ChaseTest

FactoryTest
```

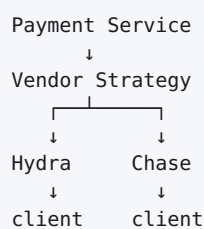
rather than one giant test.

45. A Realistic BPS Testing Strategy

Without assuming anything beyond the BPS behavior you've described, think about the layers separately:



For vendor processing:



The vendor clients can be mocked in unit tests.

The important point is:

“

Don't test the entire BPS application with one huge test. Test each layer's responsibility independently, then have a smaller number of integration tests verify that the layers work together.

46. What Should You Mock in BPS?

When you inspect your actual code, look for dependencies such as:

```
Repository
External payment client
BHS client
Factory
Other services
Message publisher
Cache
```

For a pure service unit test:

```
PaymentService
↓
mock Repository
mock external client
mock BHS client
```

Then test the service's business decisions.

Don't mock the class you're actually testing.

47. Example Interview Question

INTERVIEW SCENARIO

Interviewer:

“

“How would you unit test a service that calls an external payment service?”

Strong answer:

“

I would isolate the service from the external system by mocking the client interface. I would stub successful and failure responses from the client and verify the service's business behavior for each scenario. I'd test success, validation failure, timeout/error response, and any fallback or retry-related behavior that belongs to the service. I would also verify important interactions, such as whether the payment repository is updated, but I wouldn't over-verify every internal method call.

48. Another Important Interview Question

“How do you test a method that calls three dependencies?”

Example:

```
PaymentService
  ↓
Repository
  ↓
VendorClient
  ↓
HistoryService
```

You can:

```
@Mock
PaymentRepository repository;

@Mock
VendorClient vendorClient;

@Mock
HistoryService historyService;

@InjectMocks
PaymentService service;
```

Then:

```
Arrange
  ↓
Stub dependency behavior

Act
  ↓
service.process()

Assert
  ↓
business result
```

Verify
↓
important interactions

49. Mockito Verification vs Assertion

This distinction is subtle.

Assertion

Tests the **result/state**:

```
assertEquals(  
    PaymentStatus.PAID,  
    result.getStatus()  
);
```

Verification

Tests **interaction with a dependency**:

```
verify(repository)  
    .save(payment);
```

Use assertions for the unit's observable behavior.

Use verification when the interaction itself is part of the behavior you care about.

50. Testing Async Code

If your application uses:

```
CompletableFuture  
@Async  
ExecutorService
```

don't simply do:

```
Thread.sleep(5000);
```

in tests.

Prefer deterministic synchronization:

```
CompletableFuture<Result> future =  
    service.processAsync();  
  
Result result =  
    future.join();  
  
assertEquals(expected, result);
```

For Spring async code, test the behavior with controlled executors or appropriate integration testing rather than relying on arbitrary sleeps.

51. Test Doubles — Know the Terminology

Type	Purpose
Dummy	Passed around but not really used
Stub	Provides predefined responses
Mock	Verifies interactions
Spy	Wraps real object; selectively stubs
Fake	Working lightweight implementation

Example:

```
Stub
→ "When repository.findById(), return payment."

Mock
→ "Verify repository.save() was called."

Fake
→ In-memory repository implementation.
```

You don't need to force these terms into every interview answer, but know the distinctions.

52. Common Mistakes

△ COMMON TRAP

Mistake 1 — Testing implementation instead of behavior

```
"Was private method X called?"
```

Usually not useful.

△ COMMON TRAP

Mistake 2 — Mocking everything

If your class is pure logic:

```
Calculator calculator;
```

don't mock the calculator you're testing.

△ COMMON TRAP

Mistake 3 — Using `@SpringBootTest` everywhere

That turns simple unit tests into slow context-loading tests.

△ COMMON TRAP

Mistake 4 — Calling real external APIs

Unit tests should be deterministic.

⚠ COMMON TRAP

Mistake 5 — Depending on test execution order

Tests should be independent.

⚠ COMMON TRAP

Mistake 6 — Overusing `verify()`

Don't verify every trivial implementation detail.

⚠ COMMON TRAP

Mistake 7 — Ignoring failure paths

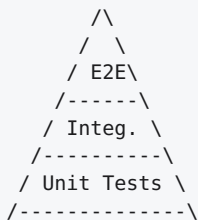
Production bugs often occur in:

```
null/empty input
exceptions
timeouts
invalid state
duplicate requests
external failures
```

not just the happy path.

53. The Testing Pyramid

A useful architecture:



Usually:

```
Many unit tests
↓
Fewer integration tests
↓
Small number of end-to-end tests
```

Why?

Unit tests are:

```
fast
cheap
isolated
```

Integration/E2E tests provide broader confidence but cost more to execute and maintain.

54. Interview-Ready Answer: “How Do You Unit Test a

Spring Service?"

INTERVIEW SCENARIO

Use this:

" I use JUnit 5 as the test framework and Mockito to isolate the service from external dependencies. I typically use `@ExtendWith(MockitoExtension.class)`, `@Mock` for dependencies, and `@InjectMocks` for the class under test. I follow Arrange-Act-Assert, stub dependency responses with `when(...).thenReturn(...)`, assert the business result, and use `verify()` for important interactions. I cover both happy paths and failure scenarios such as validation errors, missing data, dependency failures, and edge cases. I use `@SpringBootTest` or more focused Spring test slices only when I need to test Spring integration rather than isolated business logic.

55. Interview Questions You Should Be Able to Answer

JUnit 5

1. What is JUnit 5?
2. JUnit 4 vs JUnit 5?
3. `@BeforeEach` vs `@BeforeAll`?
4. `@AfterEach` vs `@AfterAll`?
5. `assertEquals` vs `assertSame`?
6. How do you test exceptions?
7. What is `@ParameterizedTest`?
8. What is `@Nested`?
9. How do you organize test setup?
10. What makes a good unit test?

Mockito

11. What is Mockito?
12. Mock vs spy?
13. `@Mock` vs `@InjectMocks`?
14. `when().thenReturn()`?
15. `thenThrow()`?
16. `verify()`?
17. `verify(..., never())`?
18. `ArgumentCaptor`?
19. `any()` vs `eq()`?
20. Why shouldn't you mock everything?
21. What is `doReturn()` and when is it useful with spies?
22. What is `verifyNoInteractions()`?

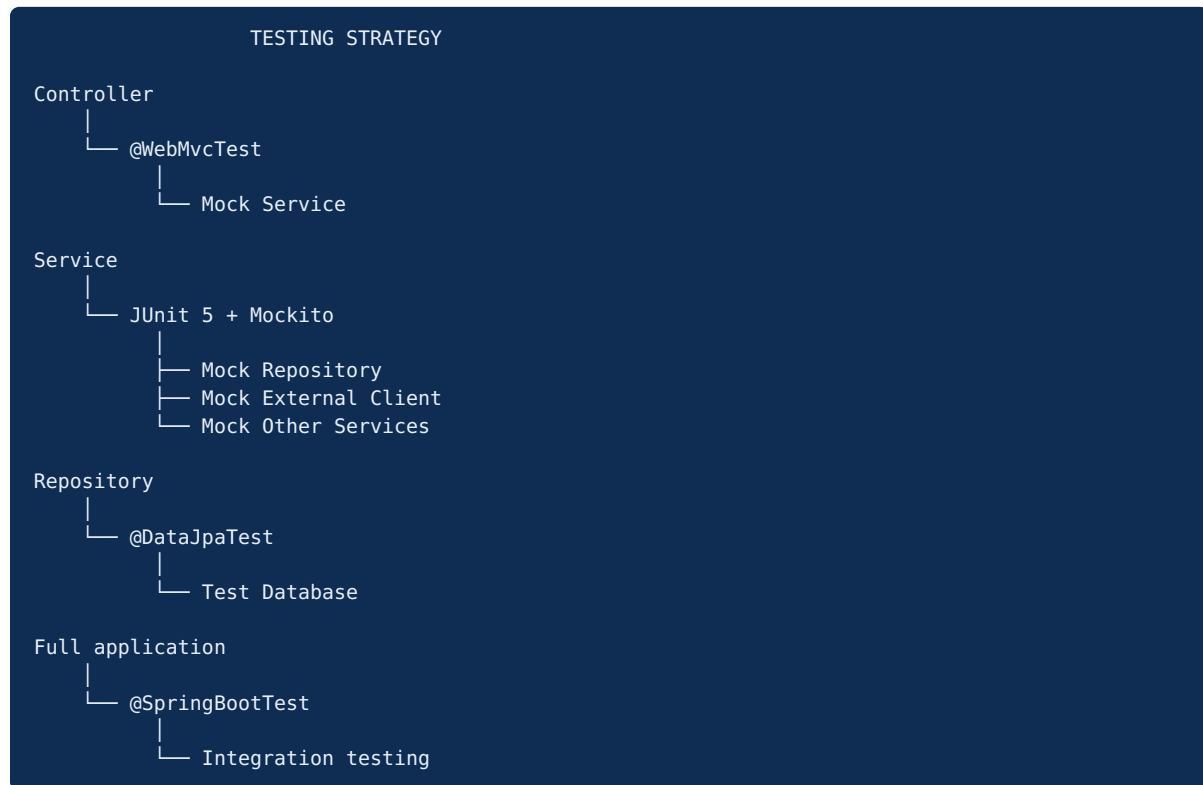
Spring Boot

23. `@SpringBootTest` vs Mockito unit test?
24. `@WebMvcTest`?
25. `@DataJpaTest`?

26. How do you test a controller?
27. How do you test a repository?
28. How do you test a service calling an external API?
29. What should be mocked?
30. What should be tested using integration tests?

56. The One Architecture You Should Remember

For a typical Spring Boot backend:



If you understand this architecture and can **write the actual tests**, you are at a good interview level for unit testing.

BPS-specific questions to investigate

When you inspect your BPS code, identify **one real test example** for each:

1. A service test using Mockito.
2. A repository test.
3. A controller test.
4. A test for Hydra/Chase failure behavior.
5. A test involving the vendor Strategy/Factory.
6. A test for an exception path.
7. A test where `ArgumentCaptor` would be useful.
8. Whether your project uses `@SpringBootTest`, `@WebMvcTest`, or `@DataJpaTest`.
9. What code coverage tool your CI pipeline uses, if any.
10. Whether tests run automatically during the PR/build pipeline.

Those examples will give you much stronger interview answers than simply saying **“I know JUnit and Mockito.”**